九阶 · Linux 与 Python 自学资料

08

监控入门

Prometheus、Grafana 与阈值告警

教科书

AYMLiu-Pzw · 2026-10-22

九阶 · Linux 与 Python 自学资料

第 08 册

监控入门

Prometheus、Grafana 与阈值告警

资料类型	教科书
适用读者	完成 Docker 阶段 4 的学习者
发布日期	2026-10-22
作者署名	AYMLiu-Pzw

阅读导航

按章级标题定位正文；PDF 阅读器中的书签同时提供章、节两级入口。

1. 5-01 监控地图：采集 / 存储查询 / 展示
2. 5-02 用 compose 起 Prometheus + Grafana + node_exporter
3. 5-03 导入面板，看懂三条曲线
4. 5-04 手写一条 PromQL：磁盘使用率
5. 5-05 配一条阈值告警规则
6. 5-06 招牌关：用 Python 拉指标，超阈值报警 + crontab

08 • 监控入门

本册是阶段 5 的正文，从 5-01 起步；配套与路线见《17-关卡总表》。
本册共 6 关（5-01 ~ 5-06）。目标：用 compose 起 Prometheus + Grafana + node_exporter，看得懂 CPU / 内存 / 磁盘三条曲线。
本册是什么：监控 —— 让机器自己汇报状态，磁盘、服务、负载三件事提前知道。
动手做实验时照这两条来：实验闭环（先记录原状态 → 操作 → 按输出 / 退出码验证 → 恢复现场）和踩坑记录位（现象 → 原因 → 解决 → 一句话总结）——不必逐关都写，做完整册回头总结一次就够。

5-01 监控地图：采集 / 存储查询 / 展示

先把三个角色分清，后面两条命令才不会是"照着抄"。

角色	是谁	负责什么
采集	`node-exporter`	住在被监控的机器上，把 CPU / 内存 / 磁盘 / 网络这些数字**变成可被抓取的接口**（默认 9100 端口）
存储与查询	`Prometheus`	按抓取间隔向 exporter 要数据，**存成时间序列**，并提供查询语言 PromQL（默认 9090 端口）。未显式配置时，抓取间隔和规则评估间隔默认都是 1 分钟；本实验显式设为 15 秒
展示	`Grafana`	连上 Prometheus，把数字**画成曲线和面板**（默认 3000 端口）

数据怎么流：`node-exporter`（采集）→ `Prometheus` 定时拉（存储查询）→ `Grafana` 画图（展示）→ 你看到的曲线。
一个比喻：exporter 是**体温计**，Prometheus 是**本实验里每 15 秒记一次数的值班员**，Grafana 是**贴在墙上的曲线图**。三者缺一不可：没有体温计就没有数，没有值班员就没人记，没有图你就看不出趋势。

完成判定

· 能说出三个组件各自负责什么 · 能说出数据从哪来到哪去（一条链说下来） · 能区分 Prometheus 的默认间隔（抓取和评估均为 1 分钟）与本实验显式设置的 15 秒 · 能说出本实验由 Prometheus 评估规则，同时知道 Grafana 也可以管理告警和通知 · 能用自己的话讲清「这一册学的东西是什么、干什么用的」（见本册开篇）

5-02 用 compose 起 Prometheus + Grafana + node_exporter

这一关是招牌动作：三个容器一起跑起来，浏览器能看到界面。下面两份文件用于第一次跑通；抄写前先预测：谁采集、谁存储、Prometheus 为什么用服务名而不是 localhost 找 exporter。跑通后再用 5-05 的时间表解释状态变化。
如果报 **`docker: unknown command: docker compose`**：说明装的是老式 docker，缺 compose 插件——先 `sudo apt install docker-compose-v2`（详见 07 册《Docker 入门》的安装说明）。
拉不动镜像？（报 `i/o timeout`）先按 **07 册《Docker 入门》的「开始之前 • 先把镜像加速器配好」**做一遍，再回来。

步骤 1：建目录，写两个文件

```
$ mkdir -p ~/lab/monitor && cd ~/lab/monitor
$ cat > docker-compose.yml <<'EOF'
services:
  prometheus:
    image: prom/prometheus:v3.15.0
    container_name: prometheus
    ports:
      - "127.0.0.1:9090:9090"
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml:ro
      - prom-data:/prometheus
    restart: unless-stopped

  node-exporter:
    image: prom/node-exporter:v1.12.1
    container_name: node-exporter
    ports:
      - "127.0.0.1:9100:9100"
    volumes:
      - /:/host:ro              # 把宿主根挂进来
    command:
      - "--path.rootfs=/host"   # 让采集器按宿主根算（否则只看到容器自己的 overlay）
    restart: unless-stopped

  grafana:
    image: grafana/grafana:13.2.4
    container_name: grafana
    ports:
      - "127.0.0.1:3000:3000"
    volumes:
      - graf-data:/var/lib/grafana
    restart: unless-stopped

volumes:
  prom-data:
  graf-data:
EOF
$ cat > prometheus.yml <<'EOF'
global:
  scrape_interval: 15s
  evaluation_interval: 15s

scrape_configs:
  - job_name: "node"
    static_configs:
      - targets: ["node-exporter:9100"]
EOF
```

解读：compose 文件就是"一声令下起三个容器"的清单。注意三点：① 三个服务在**同一个 compose 网络**里，所以 Prometheus 直接写 `node-exporter:9100` 就能找到它（**服务名可作 DNS 名**）；② `volumes` 底下那两个名字是**具名卷**，数据活过容器重启；③ 三个发布端口都写成 `127.0.0.1:宿主端口:容器端口`，默认只监听本机回环地址。**不要改成裸 `9090:9090` 直接暴露公网。**确需远程访问时，使用 VPN/私网、防火墙白名单和 Grafana 强口令；Prometheus 与 node_exporter 默认没有鉴权，不能裸暴露。

步骤 2：起来

```
$ docker compose up -d
[+] Running 4/4
 ✓ Network monitor_default   Created
 ✓ Container node-exporter   Started
 ✓ Container prometheus      Started
 ✓ Container grafana         Started

$ docker compose ps
NAME            IMAGE                          STATUS         PORTS
grafana         grafana/grafana:13.2.4         Up 8 seconds   127.0.0.1:3000->3000/tcp
node-exporter   prom/node-exporter:v1.12.1     Up 8 seconds   127.0.0.1:9100->9100/tcp
prometheus      prom/prometheus:v3.15.0        Up 8 seconds   127.0.0.1:9090->9090/tcp
```

版本快照：本节 tag 按 2026-10-21 的发布页核对（Prometheus v3.15.0、node_exporter v1.12.1、Grafana v13.2.4）。这些是固定教学快照，不追 `latest`；升级时先看发布说明、备份数据卷，再整套复测 Targets、1860 面板和告警规则。

（`docker compose ps` 的列随 Docker 版本变化：新版会多出 COMMAND / SERVICE / CREATED 列、端口列还可能带 IPv6 段——**看 STATUS 是 Up 就够。**）

解读：三个 STATUS 都是 Up 才算成功。**如果哪一个不是 Up，用 4-02 学的顺序查**：`docker compose logs 服务名` 看报错。

步骤 3：验证三个入口都能开

```
$ curl -s -o /dev/null -w "%{http_code}\n" http://localhost:9090/-/healthy
200
$ curl -s http://localhost:9100/metrics | head -3
# HELP go_gc_duration_seconds A summary of the pause duration of garbage collection cycles.
# TYPE go_gc_duration_seconds summary
go_gc_duration_seconds{quantile="0"} 3.2e-05
$ curl -s -o /dev/null -w "%{http_code}\n" http://localhost:3000/login
200
```

解读：9090 返回 200 = Prometheus 活着；9100 那堆 `# HELP` 开头的东西 = **指标本身就是一张文本表**（每行一个"指标名{标签} 值"）；3000 返回 200 = Grafana 活着（首次登录用户名密码都是 admin，会让你改密码）。

步骤 4：确认 Prometheus 真的抓到了数据

浏览器打开 `http://localhost:9090` → 顶部菜单 **Status** → **Targets**，能看到一条 node，状态是 **UP**（绿色）。**这一步是排障的分水岭**：Targets 不 UP，Grafana 里就永远是空的。（刚 up -d 的头 10~15 秒，Targets 可能还是空/红的——给它一点抓取时间，刷新一两下；一直不 UP 再排查。）

完成判定

· 三个容器都是 Up · 三个入口（9090 / 9100 / 3000）都能打开 · Status → Targets 里 node 是 UP · 能先用服务名解释为什么 Prometheus 里写 `node-exporter:9100`，再用 localhost 的失败结果说明容器内外的地址差异 · 能把三个容器关掉再起来（`docker compose down / up -d`）

回滚方式：`docker compose down` 停掉并删除容器（数据卷还在，下次起来曲线还在）；不想要数据就 `docker compose down -v`。

5-03 导入面板，看懂三条曲线

这一关的产出：一张能贴进 GitHub 的截图 —— CPU / 内存 / 磁盘三条曲线。

步骤 1：给 Grafana 接上数据源

```
浏览器打开 http://localhost:3000 （账号密码 admin / admin，会让你改密码）
左侧菜单 → Connections → Data sources → Add data source → Prometheus
URL 填：http://prometheus:9090        ← 注意：这里是容器名，不是 localhost
拉到底点 Save & test → 出现 "Data source is working" 才算通
```

解读：Grafana 自己也是一个容器，它和 Prometheus 在同一个 compose 网络里，所以用**容器名**互相访问。填 `localhost:9090` 会连不上——因为对 Grafana 容器来说，localhost 是它自己。

步骤 2：导入现成面板

```
左侧菜单 → Dashboards → New → Import
输入面板 ID：1860  （Node Exporter Full，社区里最常用的一个）
点 Load → 数据源选 Prometheus → Import
```

解读：面板不用自己画。1860 这个社区面板把 node-exporter 的指标排好了版，你要做的是**看懂它**，不是从零画。

面板打开是空的 / 显示 "No data"？ 按顺序排：① Targets 还没 UP（刚 up -d 的头 10~15 秒属正常，等它刷）；② **时间范围太窄**——Grafana 默认 Last 5 minutes，而 `rate(...[1m])` 这类查询要"窗口铺满"才有数据，把时间范围拉到 Last 1 hour 再看；③ 数据源 URL 没填成容器名 `prometheus:9090`。

步骤 3：看懂三条曲线

看什么	在哪块面板	怎么判断有问题
CPU	CPU Busy / CPU 使用率	持续 80% 以上说明有程序在死循环或压力大；只看瞬间尖峰不用慌
内存	Memory Basic / 可用内存	可用内存一路向下、Swap 开始被用，说明快不够了
磁盘	Disk Space Used Basic	**使用率持续上升就是隐患**，到 80% 就该查是谁在涨（回到 1-21 的动作）

动手制造一点波动，验证曲线是活的：

```
$ yes > /dev/null &      # 起一个吃 CPU 的进程
$ sleep 60 && kill %1    # 一分钟后停掉
（回到 Grafana 刷新，看 CPU 曲线那一分钟是不是抬起来了）
```

步骤 4：截图存档

把带三条曲线的面板截图，存成 `~/lab/monitor/面板截图.png`，记下这次的改动。以后做综合项目（阶段 7）时，这张图就是"我会监控"的证据。

完成判定

· 数据源测试通过（Data source is working） · 1860 面板导入成功且曲线在动 · 能说出三条曲线各自"什么样算不正常" · 用 `yes` 制造波动并在曲线上看到 · 有一张截图存档

回滚方式：只停本终端刚启动的后台作业 `kill %1`。若作业编号已丢失，先 `pgrep -a yes` 核对 PID、命令和所有者，再按明确 PID 处理；不要直接批量杀所有同名进程。

5-04 手写一条 PromQL：磁盘使用率

这一关的目标：不靠现成面板，自己能写出一条查询 —— 因为下一关的告警规则、再下一关的 Python 报警脚本，用的都是这一条。

步骤 1：先认识两个指标名

```
浏览器打开 http://localhost:9090 → 顶部 Query 框里输入：
node_filesystem_size_bytes          → Execute

结果（节选）：
node_filesystem_size_bytes{device="/dev/sda1", fstype="ext4", mountpoint="/"}  21474836480
node_filesystem_size_bytes{device="/dev/sdb1", fstype="ext4", mountpoint="/data"}  107374182400
node_filesystem_avail_bytes{device="/dev/sda1", fstype="ext4", mountpoint="/"}  15032385536
```

解读：指标名告诉你"这是什么"，**花括号里是标签（labels）**——这一条数据属于哪块盘、哪个挂载点。**一个指标名后面带着很多条数据，靠标签区分。**这就是"时间序列"的意思。

步骤 2：算出使用率

```
100 - (node_filesystem_avail_bytes / node_filesystem_size_bytes * 100)

结果（只看 / 那一行）：
{device="/dev/sda1", mountpoint="/"}   29.99    （示例：设备名、数值都随机器）
```

解读：**可用 ÷ 总量 × 100 是"还剩百分之几"，用 100 减一下才是"用了百分之几"**（这一步最容易写反，写完一定要核对一眼数字合不合理）。**和 df 对不上？正常**——ext4 默认给 root 预留 5% 空间：df 的 Use% 往往不把那 5% 算进"已用"，而这条公式（100 − avail/size×100）会把它算进去；差 2~5 个点是口径差，不是公式写反。

步骤 3：只看想看的盘

```
100 - (node_filesystem_avail_bytes{mountpoint="/"} / node_filesystem_size_bytes{mountpoint="/"} * 100)

# 再看一下时间范围：把查询框上方的 Graph 页签点开，选 Last 1 hour
# 曲线会随时间变化 —— 这就是"趋势"，比瞬间值有用得多
```

解读：`{mountpoint="/"}` 是**过滤条件**（只留下根分区）。生产上你还会看到只查 `fstype!="tmpfs"` 之类的写法，意思是"排除临时文件系统"。

步骤 4：把这条查询放进 Grafana

```
Grafana → Dashboards → New → New panel → 查询框里粘贴同一句 → Run query
标题改成"根分区使用率" → Save
```

解读：**同一句话，在 Prometheus 里验证、在 Grafana 里展示。**自己写查询并放入面板，便于理解每个标签和计算结果从哪里来；导入现成面板只适合辅助观察。

完成判定

· 能说出"指标名 + 标签"是什么关系 · 能默写出这条使用率公式并解释为什么用 100 减 · 会用 `{mountpoint="/"}` 过滤 · 能把这条查询放进 Grafana 面板

回滚方式：没有破坏性操作；实验面板删掉即可。

5-05 配一条阈值告警规则

步骤 1：规则写在单独一个文件里

```
$ cd ~/lab/monitor
$ cat > alerts.yml <<'EOF'
groups:
  - name: disk
    rules:
      - alert: DiskUsageHigh
        expr: 100 - (node_filesystem_avail_bytes{mountpoint="/"} / node_filesystem_size_bytes{mountpoint="/"} *
100) > 85
        for: 1m
        labels:
          severity: warning
        annotations:
          summary: "根分区使用率超过 85%"
          description: "当前 {{ $value }}%，请检查是谁占的（df -h / du）"
EOF
```

解读：expr 就是上一关那条查询；> 85 是阈值；for: 1m 表示"**连续 1 分钟都超过才算真告警**"（避免瞬间尖峰乱报）。{{ $value }} 会把当时的数字填进告警文案。

参数	本实验值	管什么
抓取间隔 scrape_interval	15 秒	多久向 exporter 收一次新样本
持续条件 for	1 分钟	条件要连续满足多久，才算 FIRING
评估间隔 evaluation_interval	15 秒	多久重新计算一次规则；未显式配置时默认 1 分钟

预测：阈值改成 1 后，Prometheus 不是立刻 FIRING；它先标 PENDING，至少连续满足 for 后进入 FIRING。抓取样本、条件成立、进入 FIRING 是三个不同动作。

步骤 2：让 Prometheus 读到它

```
$ cat > prometheus.yml <<'EOF'
global:
  scrape_interval: 15s
  evaluation_interval: 15s

rule_files:
  - /etc/prometheus/alerts.yml      # ← 新增这一行

scrape_configs:
  - job_name: "node"
    static_configs:
      - targets: ["node-exporter:9100"]
EOF
$ # 把规则文件和配置都挂进容器（改 docker-compose.yml 的 prometheus 段）
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml:ro
      - ./alerts.yml:/etc/prometheus/alerts.yml:ro
$ docker compose up -d --force-recreate prometheus
[+] Running 1/1
 ✓ Container prometheus  Started
```

步骤 3：看它有没有生效

```
http://localhost:9090 → 菜单 Alerts
状态：INACTIVE（没超阈值，正常）

# 造一次真告警：把阈值临时改成 1
$ sed -i 's/> 85/> 1/' alerts.yml && docker compose up -d --force-recreate prometheus
再刷新 Alerts 页面：状态变成 PENDING → 等约 75 秒 FIRING（红色）（for: 1m + 本实验显式配置的 15 秒评估间隔；若未配
evaluation_interval，则默认每 1 分钟评估）
```

解读：**PENDING = 条件满足但还没到 for 规定的时间；FIRING = 规则处于触发状态。**这个"先 PENDING 再 FIRING"的机制用来压掉抖动误报。**本实验明确选择由 Prometheus 评估规则；Grafana 也支持 Grafana-managed alert rules，两者不是“谁唯一正确”。**另外，本实验没有 Alertmanager 或 Grafana contact point，**FIRING 不等于通知已经送达任何人**；本关只验证规则状态，不验证通知投递。

别忘了把阈值改回 85（`sed -i 's/> 1/> 85/' alerts.yml`），再用 **--force-recreate** 重建一次并**确认状态回到 INACTIVE**，把这次实验写进踩坑记录——**为什么不用 restart**：`sed -i` 会换文件本体，单文件挂载下容器里读到的还是旧内容。

完成判定

· 规则文件写对，Prometheus 启动无报错 · Alerts 页面能看到这条规则 · 阈值改成 1 后先预测、再实际看到 PENDING → FIRING · 能分别说出抓取间隔 15 秒、`for: 1m`、规则评估间隔各管什么 · 改回 85 并确认状态回到 INACTIVE

回滚方式：`sed` 改回原阈值并 `docker compose up -d --force-recreate prometheus`；规则文件备份一份到 `~/lab/monitor/alerts.yml.bak` 更稳妥。

5-06 招牌关：用 Python 拉指标，超阈值报警 + crontab

为什么这是招牌关：它把 **Python + 监控 + 定时任务** 三条线接成一个能讲的作品 —— 有真实数据源、有判断逻辑、有日志、有定时执行。这是你作品里辨识度最高的一项。

步骤 1：先看清 Prometheus 的返回长什么样（这一步不看懂，后面必卡）

```
$ curl -s 'http://localhost:9090/api/v1/query?query=100-
(node_filesystem_avail_bytes%7Bmountpoint%3D%22/%22%7D/node_filesystem_size_bytes%7Bmountpoint%3D%22/%22%7D*100)
'
{
  "status": "success",
  "data": {
    "resultType": "vector",
    "result": [
      {
        "metric": {"device":"/dev/sda1","fstype":"ext4","mountpoint":"/"},
        "value": [1759999999.123, "42.31"]
      }
    ]
  }
}
```

逐层解读（这就是卡点所在）：

路径	是什么
`status`	`success` 才说明查询本身没问题（写错查询会是 `error` 并带 `errorType`）
`data.resultType`	返回的数据形状：`vector`=某个瞬间的一组值；`matrix`=一段时间的序列。我们要的是 vector
`data.result`	**一个列表**（因为可能匹配到多条序列：多块盘、多个挂载点）
`result[0].metric`	这条数据的标签（哪块盘、哪个挂载点）
`result[0].value`	一个两元素列表：**[时间戳, "数值字符串"]**。注意数值是**字符串**，用之前要 `float()` 一下

一句话记住：**status → data → result（列表）→ [0] → value[1] 才是那个数字。**查不到数据时 `result` 会是空列表 `[]`，脚本必须能扛住这种情况。

步骤 2：写脚本（可原样复制，只改开头的地址）

依赖前置：脚本使用第三方库 `requests`。系统 Python 路线先执行 `sudo apt install -y python3-requests`；若按本套书约定使用 venv，则执行 `python3 -m venv .venv`、`source .venv/bin/activate`、`pip install requests`。装好后先运行一次 `python3 -c "import requests; print(requests.__version__)"`。

```
#!/usr/bin/env python3
"""查 Prometheus 的磁盘使用率，超阈值就报警。用法：./disk_watch.py --threshold 85"""
import argparse
import logging
import os
import sys

import requests

QUERY = ('100 - (node_filesystem_avail_bytes{mountpoint="/"} '
         '/ node_filesystem_size_bytes{mountpoint="/"} * 100)')


def get_usage(url):
    """返回 (使用率, 标签)；拿不到数据就抛异常，由调用方处理"""
    resp = requests.get(f"{url}/api/v1/query", params={"query": QUERY}, timeout=5)
    resp.raise_for_status()               # 非 200 直接抛出来
    payload = resp.json()
    if payload.get("status") != "success":
        raise RuntimeError(f"查询失败：{payload.get('error')}")
    result = payload["data"]["result"]
    if not result:                        # 空列表 = 没数据（例如挂载点写错）
        raise RuntimeError("查询没有返回任何数据")
    labels = result[0]["metric"]
    usage = float(result[0]["value"][1])  # 注意：字符串，要转 float
    return usage, labels


def main():
    parser = argparse.ArgumentParser(description="磁盘使用率报警")
    parser.add_argument("--url", default="http://localhost:9090", help="Prometheus 地址")
    parser.add_argument("--threshold", type=float, default=85, help="告警阈值（百分比）")
    args = parser.parse_args()

    logging.basicConfig(filename=os.path.expanduser("~/lab/monitor/disk_watch.log"), level=logging.INFO,
                        format="%(asctime)s %(levelname)s %(message)s")
    try:
        usage, labels = get_usage(args.url)
    except Exception as exc:                                  # 网络/HTTP/JSON/结构 全都兜住
        logging.error("拉取指标失败：%s", exc)
        print(f"[错误] 拉取指标失败：{exc}")
        return 1

    logging.info("阈值=%.1f 当前=%.2f 标签=%s", args.threshold, usage, labels)
    if usage >= args.threshold:
        logging.warning("磁盘告警：%.2f%% 超过阈值 %.1f%%", usage, args.threshold)
        print(f"[告警] 磁盘使用率 {usage:.2f}%，超过阈值 {args.threshold}%")
        return 2
    print(f"[正常] 磁盘使用率 {usage:.2f}%")
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

解读（三处关键）：① `timeout=5` 必须有 —— 否则 Prometheus 挂了脚本会一直吊着；② `float(result[0]["value"][1])` 是因为返回的数值是字符串；③ **退出码分了三种**（0 正常 / 1 出错 / 2 告警），这样 crontab 和别的脚本都能判断结果。

步骤 3：跑三次，验证三种情况

（示例：设备名、数值随机器和时间变化。）

```
$ chmod +x disk_watch.py
$ ./disk_watch.py
[正常] 磁盘使用率 42.31%
$ ./disk_watch.py --threshold 1
[告警] 磁盘使用率 42.31%，超过阈值 1.0%
$ ./disk_watch.py --url http://localhost:9999      # 故意写错端口
[错误] 拉取指标失败：HTTPConnectionPool(host='localhost', port=9999): Max retries exceeded ...
$ tail -3 ~/disk_watch.log
2026-10-09 21:10:03,221 INFO 阈值=85.0 当前=42.31 标签={'device': '/dev/sda1', ...}
2026-10-09 21:10:11,887 WARNING 磁盘告警：42.31% 超过阈值 1.0%
2026-10-09 21:10:19,004 ERROR 拉取指标失败：HTTPConnectionPool(...)
```

解读：三种情况都有明确提示、日志都留痕，这就叫"能交给别人的脚本"。

步骤 4：挂进 crontab，每 5 分钟跑一次

```
$ crontab -e
*/5 * * * * /usr/bin/python3 /home/你的真实用户名/lab/monitor/disk_watch.py >> /home/你的真实用户名/lab/monitor/disk_watch.log 2>&1

$ crontab -l                       # 确认写进去了
$ sleep 300 && tail -2 ~/lab/monitor/disk_watch.log   # 等一轮，看脚本自己写入的指标或告警
```

解读：**路径全部写绝对路径**（把行里的 /home/你的真实用户名 换成你自己的；cron 环境不展开 ~，1-35 学过的坑）；`>> ... 2>&1` 把输出和错误都追加进日志。系统 Python 路线写 `/usr/bin/python3`；venv 路线写 venv 解释器的绝对路径，例如 `/home/你的用户名/lab/monitor/.venv/bin/python /home/你的用户名/lab/monitor/disk_watch.py`，不要只写 `.venv/bin/python`，也不要把脚本写成家目录根下的旧路径。

完成判定

· 能画出返回结构（status → data → result → [0] → value[1]）并说出哪一步是数值 · 三种情况（正常 / 告警 / 连不上）都跑过，都有明确提示 · 日志文件里三种记录都有 · `crontab` 里挂了任务，日志能看到脚本自己的 INFO/WARNING/ERROR 记录（不是 Python 找不到文件） · 能说清退出码 0 / 1 / 2 各自什么意思

回滚方式：先 `crontab -l` 核对现有任务，再进入 `crontab -e`，只删除本次新增的那一行。需要留底可先把 `crontab -l` 输出保存到明确文件。`crontab -r` 会删除该用户全部定时任务，不作为默认回滚；脚本和日志都在 `~/lab/monitor/` 下，确认不再需要后再删。

阶段 5 • 学习总结练习：把监控接成一条"能报警"的链子

业务场景：领导说"我不想天天问人机器好不好，我要它自己会喊"。你要交付的不是一张截图，而是**一条从采集到报警的完整链路。**

综合要求：① 用 compose 起 Prometheus + Grafana + node_exporter 三件套；② 在 Grafana 上有一块**自己写 PromQL** 的面板（磁盘使用率，不是导入的现成面板）；③ 配一条告警规则，把阈值临时改到 1% 并**看到 PENDING → FIRING**；④ 用 Python 脚本调 Prometheus HTTP API 拿同一个指标，超阈值写日志告警，并挂进 crontab；⑤ 把"手工看的曲线"和"脚本报的警"两条路都截图留档。

验收：· 三个容器都是 Up、Targets 里 node 这条 job 是 UP · 面板上的曲线会动（能用手造负载验证） · 告警规则真的从 PENDING 走到 FIRING · 脚本能拉到真实指标；阈值调 1% 能触发；日志有记录 · 三张截图（面板 / 告警状态 / 脚本输出）进仓库

产出：仓库 `monitor-demo`（compose + prometheus.yml + alerts.yml + `disk_watch.py` + 截图 + README）

AI 运维指向：这就是 AI 集群可观测性的最小版 —— 把 node_exporter 换成 GPU exporter（8-08 做过），链条一模一样。

本册复习卡

① exporter 采集、Prometheus 定时抓取并查询、Grafana 展示；告警由规则的 `expr + for` 判定。

② 看到 PENDING → 先查当前值是否持续超阈值和 `for` 是否满足；看到 DOWN → 先查 Targets 与 exporter 端口。

③ 独立变式：把抓取间隔改为 30 秒、`for` 改为 2 分钟，按时间表估算从条件成立到可能 FIRING 的最早时间，再实际验证；不附完整操作步骤。